Programmation sur Processeur Graphique – GPGPU
TD 2 : allocation mémoire, kernel et erreurs

Centrale Nantes
P.-E. Hladik, pierre-emmanuel.hladik@ec-nantes.fr
—
Version 1.0.0 (23 janvier 2025)

1 Le b.a.-ba du CUDA

Vous allez reprendre le code d'addition de vecteur pour vous exercer.

Objectif 1.1

— utiliser `cudaMemcpy` et `cudaMalloc`
— lancer un kernel

(1.1) Travail à faire : Addition vectorielle

1. Récupérer le code source `addvec.cu`.
2. Modifiez le pour qu'il réalise le travail demander, pour cela il ne faut modifier que les `/* FIXE ME */`.
3. Compiler, exécuter et observer.
4. Modifier le code (valeur de N ligne 3 dans le code initial) pour prendre en considération une taille quelconque des vecteurs (et non pas simplement un multiple de 512).

(1.1) Comment faire : recopie de la mémoire

La documentation de `cudaMemcpy` est disponible sur

https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY.html

(1.2) Comment faire : Lancement d'un kernel

Le lancement d'un kernel se fait comme un appel normal à une fonction en ajoutant des paramètres de configuration d'exécution entre <<< et >>> avant les arguments de la fonction. Le premier argument donne le nombre de blocs et le second le nombre de threads par bloc.

2 Des bonnes pratiques : gestion des erreurs

Objectif 2.1

— gestion des erreurs synchrones

(2.1) Travail à faire : Gestion des erreurs

Toutes les fonctions de l'API CUDA renvoient un code d'erreur de type `cudaError_t`. Si l'appel ne retourne pas d'erreur cette valeur est égale à `cudaSuccess`.

Le fichier `error.h` fourni avec le sujet propose une macro (voir ci-dessous ce qu'est une macro) pour tester tous les appels aux fonctions CUDA. La macro permet de générer à la compilation le code pour ajouter un test sur la valeur de retour de la fonction (et donc sur le succès ou non de la fonction). Pour pouvoir utiliser la macro il suffit d'inclure `error.h` dans votre fichier.

Utiliser cette macro pour tester le code d'erreur de retour pour chaque appel à l'API dans le code `error.cu` (voir ci-dessous comment l'utiliser dans un code).

(2.1) Comment faire : C'est quoi une macro en C

Une macro en C est une construction définie par le préprocesseur qui permet de remplacer du code avant la compilation. Les macros sont définies avec le mot-clé `#define` et servent généralement à simplifier le code, améliorer sa lisibilité ou définir des constantes. Elles ne sont pas des fonctions ou des variables, elles sont simplement remplacées directement dans le code source avant la compilation, par exemple, `#define PI 3.14159`. il est aussi possible de passer des arguments à une macro, par exemple `#define SQUARE(x) ((x) * (x))`.

(2.2) Comment faire : Créer un affichage lisible

La fonction `cudaGetErrorString(cudaError_t err)` convertit un code d'erreur en une chaîne de caractères, il peut être utilisé par exemple

```
printf("%s in %s at line %d\n", cudaGetErrorString(err), __FILE__, __LINE__);
exit(EXIT_FAILURE);
```

(2.3) Comment faire : Passer un paramètre à une macro

Une macro avec paramètres s'écrit `#define nom (<liste_paramètres>) macro`, par exemple :

```
#define min(a, b)       ((a) < (b) ? (a) : (b))
```

(2.4) Comment faire : Ecriture d'une macro sur plusieurs lignes

Il est possible d'écrire une macro sur plusieurs lignes en ajoutant \ (backslash) à la fin de la ligne, par exemple :

```
#define MACRO_EXPORT (target)                    \
  #ifdef _MSC_VER                                \
    #if defined TARGET_EXPORTS                   \
      #define TAGET_API __declspec( dllexport ) \
    #else                                        \
      #define TARGET_API __declspec( dllimport )\
  #endif // _LIB
```

(2.5) Comment faire : Comment utiliser la macro `CHECK_ERROR`

La macro `CHECK_ERROR` prend en paramètre une fonction CUDA (et donc son retour d'erreur), par exemple pour vérifier le retour d'erreur de la fonction `cudaFree(a);`, il faut écrire `CHECK_ERROR(cudaFree(a));`.

3 Des bonnes pratiques : gestion des erreurs (le retour)

Objectif 3.1

— gestion des erreurs asynchrones

(3.1) Travail à faire : Gestion des erreurs asyncrhones

L'utilisation de `<<< ... >>>()` n'est pas un appel à l'API CUDA et ne renvoie pas de code d'erreur. De plus il est asynchrone, ce qui signifie que le thread émet une requête pour exécuter le kernel sur le GPU, puis continue sans attendre que le kernel soit terminé.

Une erreur peut être capturée avec `cudaGetLastError()` [a] après le lancement du noyau, mais il peut arriver qu'elle ne soit pas encore signalée au moment de l'appel (le kernel étant en cours d'exécution).

Il est possible de capturer l'erreur asynchrone en synchronisant explicitement l'exécution du noyau CUDA avec le thread, par exemple en utilisant `cudaDeviceSynchronize` [b]. Cependant, la synchronisation explicite affecte les performances et le comportement de l'exécution, il n'est donc pas recommandé de l'utiliser en production. Voir ci-dessous une utilisation possible de ces fonctions.

Ajouter les éléments nécessaire pour vérifier les erreurs de l'appel au kernel avec l'exemple fourni dans le fichier `error2.cu`.

a. https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__ERROR.html#group__CUDART__ERROR_1g3529f94cb530a83a76613616782bd233

b. https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__DEVICE.html#group__CUDART__DEVICE_1g10e20b05a95f638a4071a655503df25d

(3.1) Comment faire : Exemple pour capturer une erreur asynchrone

Exemple d'utilisation de `cudaGetLastError` avec `cudaDeviceSynchronize` :

```
dev<<<...>>>(...);
ret = cudaGetLastError();
if (debug) ret = cudaDeviceSynchronize();
```

Remarquez que `cudaDeviceSynchronize` renvoie une erreur si l'une des tâches précédentes a échoué.

4 Bien gérer sa mémoire

Objectif 4.1

— approfondissement sur l'allocation de la mémoire

(4.1) Travail à faire : Manipulation mémoire

Le code source `mesh.c` fournit des fonctions pour manipuler un objet 3D décrit au format OBJ. Une surface polygonale est décrite par un ensemble de sommets (vertices) et un ensemble de faces composés de ces sommets.

Un objet est décrit dans la structure `object_t` qui comprend quatre champs :
— `int nb_vertices` : le nombre de sommets de l'objet
— `vertex_t *vertex_t` : un pointeur vers un tableau de `vertex_t`
— `int nb_faces` : le nombre de faces
— `face_t * faces` : un pointeur vers un tableau de `face_t`

De même, `vertex_t` et `face_t` sont des structures contenant les informations de position des sommets et des faces.

Le code actuel permet d'importer un fichier OBJ et d'appliquer une transformation (`flip_and_expand`) sur cet objet. La transformation inverse les faces et applique un facteur sur les coordonnées des sommets.

Vous devez passer en CUDA ce code pour appeler la fonction `flip_and_expand` sur le GPU. Ne cherchez pas à paralléliser cette fonction, mais simplement à l'appeler en la passant en `__global__` dans un unique thread (`flip_and_expand<<<1,1>>>`). **La difficulté de cet exercice est l'utilisation de la mémoire.** N'oubliez pas de changer l'extension du fichier en `.cu` pour pouvoir le compiler avec `nvcc`.

Si vous êtes en avance et vous vous ennuyez, vous pouvez commencer à paralléliser le traitement d'un mesh...